Benson Cordeiro, Dharshan Amin, Tanay Jawle
bbc6, dhamin2, tjawle2
Spring 2025 - Final Report
IS 455- Database Designing and Prototyping

Final Report - Asynchronous Task Manager

1. Problem Definition

In large-decentralized organizations, managing tasks efficiently across various teams and hierarchies often leads to organizational and scheduling conflicts. Managers frequently face difficulties in coordinating responsibilities, tracking progress and ensuring timely task completion, especially when teams are spread across different locations and work structures. Our asynchronous task manager aims to streamline task delegation by allowing managers to assign tasks only to team members below them in the organizational hierarchy, which ensures clarity in responsibilities and avoiding cross-level assignment conflicts. By centralizing task management and enforcing structured delegation the system enhances overall efficiency, accountability, and team collaboration within complex organizational setups.

2. E-R Diagram

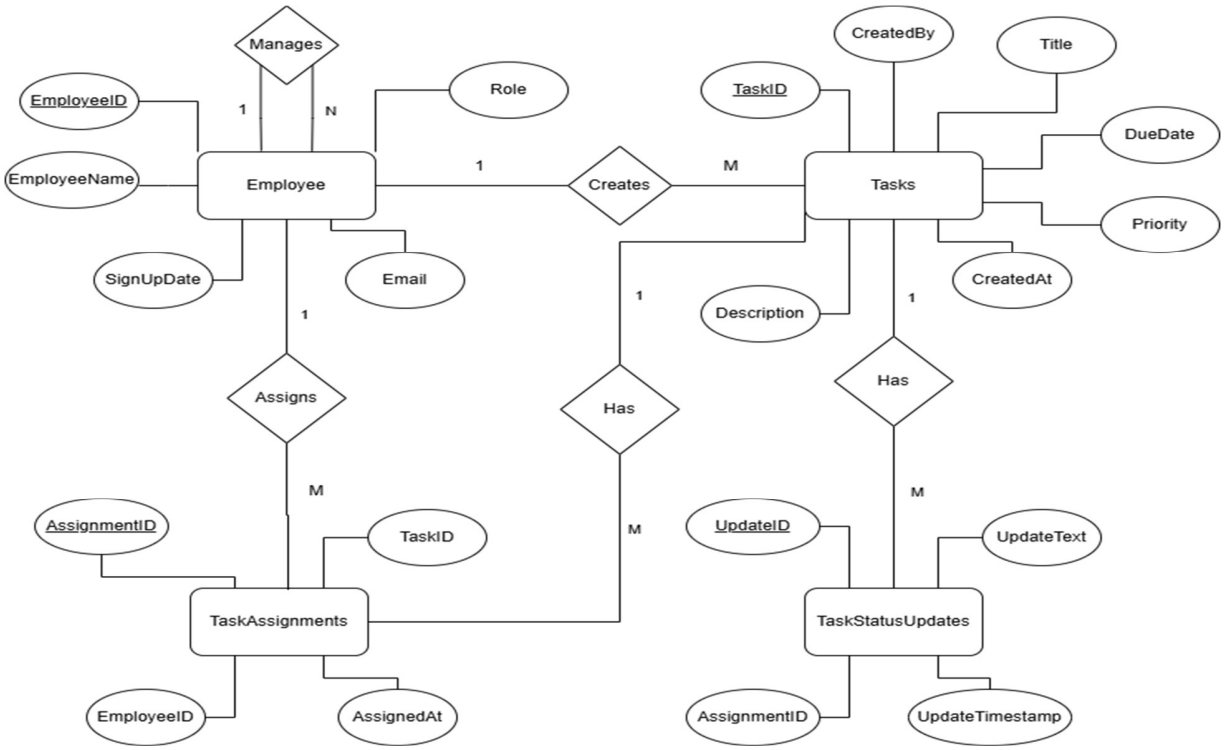


Figure 2.1: ER Diagram

Figure 2.1 is the entity relationship diagram that shows how employees and tasks are connected in our system. Each employee has basic details like name, email, signup date,

and role. An employee can manage multiple other employees, and they can also create many tasks. Each task has details like title, due date, priority, description, and the employee who created it. Tasks can be assigned to employees through task assignments, which record when the assignment was made. Tasks can also have multiple status updates, each with a timestamp and text. This ensures that the system is scalable and uses as less space as possible.

3. Relational Schema

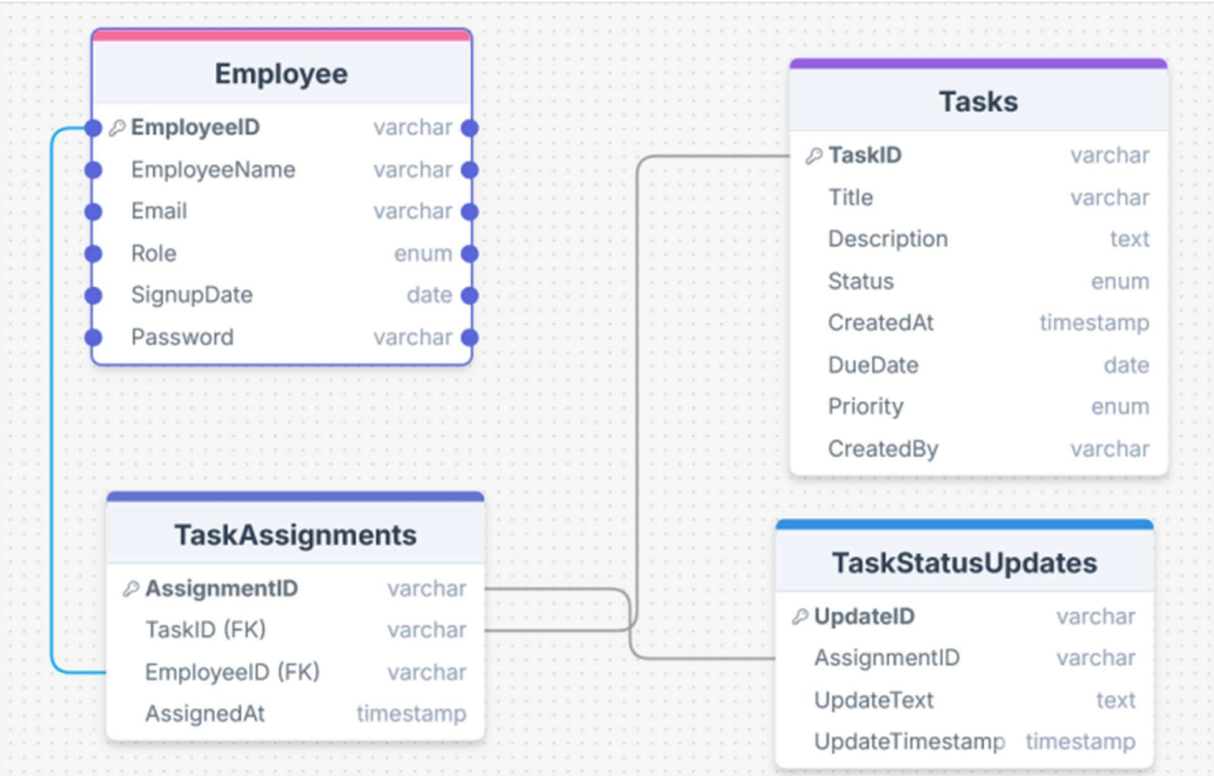


Figure 3.1: Relational Schema

Figure 3.1 is relational schema that shows how data across four connected tables using keys and specific data types to keep everything structured and efficient. The Employee table uses EmployeeID as its primary key and stores important attributes like EmployeeName and Email as varchar, Role as an enum (for example: Manager or Employee), SignupDate as a date, and Password as varchar. The Tasks table has TaskID as the primary key and includes Title and CreatedBy as varchar, Description as text, Status and Priority as enums, CreatedAt as a timestamp, and DueDate as a date. The TaskAssignments table uses AssignmentID as the primary key, and both TaskID and EmployeeID are foreign keys linking back to the Tasks and Employee tables. AssignedAt is stored as a timestamp. Finally, the TaskStatusUpdates table has UpdateID as the primary

key, with AssignmentID as a foreign key referencing TaskAssignments. It also contains UpdateText as text and UpdateTimestamp as a timestamp. These primary and foreign keys maintain relationships between the tables and ensure that data like tasks, employees, and status updates are correctly linked.

We have designed our relational schema with a clear goal of achieving at least Third Normal Form (3NF) to ensure that the data remains clean, efficient, and free from redundancy. Following the normalization process, we first ensured First Normal Form (1NF) by organizing our data into flat tables with atomic values. For example, rather than storing a list of tasks within the Employee table, we introduced the TaskAssignments table. This allows each task assignment to be recorded as a separate row, making the structure more organized and easier to query or update.

To meet Second Normal Form (2NF), we eliminated partial dependencies by assigning each table a single, unique primary key. In our schema, TaskAssignments has its own AssignmentID as the primary key, rather than using a composite key like EmployeeID and TaskID. This prevents attributes like AssignedAt from depending on just one part of a composite key. Additionally, we avoid storing employee-related details such as EmployeeName or Email inside TaskAssignments, since those belong in the Employee table, which holds all employee-specific data tied to EmployeeID.

Finally, we followed the rules of Third Normal Form (3NF) by removing transitive dependencies. In our Tasks table, we only include CreatedBy as a foreign key that points to the EmployeeID, and we do not include redundant data such as the employee's name or role there. That information can be retrieved through a join with the Employee table when needed. By separating data in this way, we avoid duplication, reduce the risk of inconsistencies, and make our system easier to maintain or scale. This thoughtful structure helps us achieve the clarity and efficiency that normalization is meant to provide.

4. Integrity Constraints

Entity integrity ensures that every row within a table is able to be uniquely distinguished. This is accomplished through having a primary key for each table, i.e., EmployeeID for the Employee table, TaskID for the Tasks table, and AssignmentID for the TaskAssignments table. The primary keys should always be populated and should never remain empty. By enforcing entity integrity, the system prevents any two records in a single table from being identical and ensures that all units of information are traceable to a single unique entry. This is necessary to prevent duplicate data and provide a well-organized database.

Referential integrity is the upkeep of consistencies between relations between tables. It requires that every foreign key value of one table is referenced with an authorized primary

key value within the referencing table. For example, the CreatedBy value within the Task table must point to an existing EmployeeID within the Employee table. Every record in the TaskAssignments table should pair a valid EmployeeID and TaskID with their corresponding tables, respectively. This constraint prevents dangling references in the data, i.e., when a task is assigned to an employee who is not present in the system, such that all associations in the database are correct and trustworthy.

Domain integrity is about data integrity of being stored in a column with the right data types and valid range of values. Each field of a table follows some rules stating what kind of information the field can hold. For instance, the Role column of the Employee table can be given only a fixed set of roles like Manager or Employee, and the Status column of the Tasks table can be given values like 'Pending', 'In Progress', or 'Completed'. Such restrictions avoid improper or inconsistent input which makes the data always meaningful and consistent throughout the database.

5. Security Design

To ensure a safe and organized user experience, our system employs a role-based access control system to restrict user access based on the roles they have been assigned within the organization. Through this structure, such unauthorized access to sensitive data is prevented, and users only have access to features that are dedicated to their tasks. The roles established in our system include:

1. **Upper Senior Manager:** This role has the highest level of access in the system. Upper Senior Managers can create, assign, and see all tasks across the organization and therefore can see the task assignment process at the organizational level.
2. **Manager:** Managers are able to delegate tasks to employees within their team and view tasks that have been delegated to them. This role is specifically disallowed from delegating tasks to those above their level (e.g., a Manager cannot delegate tasks to an Upper Senior Manager), maintaining clear hierarchical boundaries.
3. **Employee:** Employees have the minimum level of access, as they can only view tasks allocated to them. This ensures that employees are solely concentrated on their own work without being exposed to broader organizational data.

Role assignment is handled safely in the backend such that only administrators can assign or modify user roles. The system ensures that the role of every user is stored safely in the database, and role-based permission is checked whenever a user operates the system.

For an additional level of protection, passwords are hashed using bcrypt before storing them in the database, so user credentials are never stored in plaintext. This hashing significantly reduces the likelihood of password stealing in the event of a breach.

In addition, front-end role checks restrict access to routes. For example, an Employee user will not be able to access routes designated for Managers or Upper Senior Managers so users will only be able to see and work with data relevant to what they are assigned for.

Besides, user management functionality is included at the backend level, where an administrator can create and manage users, assign roles, and grant secure access to the system. Role assignment to users can be performed only by the administrator, so no unauthorized change in user access levels can be performed.

Through this structured security design, we offer secure role-based access to information and activities, preventing unauthorized access and possessing clearly defined boundaries of access based on organizational hierarchy.

6. Application Functional Design

The application we designed has a robust set of functionalities to streamline task management across hierarchical roles within the organization. Below is an overview of the main features implemented with a detailed walkthrough of how we used query operations and integrated it within each function:

1. **Login Authentication**
   The login functionality ensures secure access to the system, requiring users to authenticate with hashed credentials. Passwords are hashed using bcrypt before storage in the database. Once authenticated the system checks the user's role and grants access to the appropriate features. The query operation here checks the user’s credentials and role from the database before granting access to the system.

2. **Create Task**
   This feature allows Managers and above (including Upper Senior Managers) to create new tasks. Users can input task details like title, description, priority and due date. When the task is created it is added to the database and an initial status (such as "Pending") is assigned. This is the add operation where a task is added to the system. The task is then assigned to the appropriate team member based on their role and eligibility.

3. **Assign Task**
   Managers can assign tasks to eligible employees based on role hierarchy. This ensures that tasks are assigned within a specific team and that no task is assigned across hierarchical levels (i.e., managers assigning tasks to other managers). The modify operation is used when tasks are assigned to employees—updating the task with the assigned users and relevant deadlines. When a task is successfully assigned it is added to the list of tasks of the specific employees and the task appears in their dashboard under ‘My Tasks.’

4. **Update Task**
   The Update Task functionality allows users to modify task details such as status, priority, description or deadlines. For example, a manager might change the status of a task from "Pending" to "In Progress" or update the priority based on changing work requirements. This represents the modify operation within the system, where existing tasks are updated in the database. The query operation is used to retrieve tasks for modification, ensuring that the correct task is being updated based on the user's request.

5. **View Tasks**
   The View Tasks feature provides various views based on user roles:

   - **My Tasks**: Displays tasks assigned to the logged-in user. Employees can view their personal tasks, while Managers and Upper Senior Managers can monitor their team members' progress. This is a query operation where tasks are filtered based on the logged-in user's role.

   - **All Tasks**: Available only to Managers and Upper Senior Managers, this view allows them to see all tasks assigned within their teams or across the organization. It helps managers oversee the task delegation and monitor team productivity. This too is a query operation, allowing managers to filter tasks by status, priority, or assignee.

6. **Role-Specific UI**
   The Role-Specific UI ensures a dynamic user experience, changing based on the logged-in user's role. The system adjusts the interface by displaying personalized welcome messages, role-appropriate theme colors, and visible tabs for each user. This dynamic customization ensures that users only see what's relevant to them based on their access rights. For example, a Manager will see additional task management features while an Employee will only see the "My Tasks" tab. The modify operation is applied here to update the UI dynamically based on role, and query operations are used to retrieve role-specific information that drives the UI adjustments.

Benson Cordeiro, Dharshan Amin, Tanay Jawle
bbc6, dhamin2, tjawle2

7. System Implementation

1. Database

AssignmentID	TaskID	EmployeeID	AssignedAt
A11	T11	E18	2025-04-29 00:42:28
A110	T108	E20	2025-05-01 23:08:30
A111	T112	E18	2025-05-04 17:34:54
A112	T108	E5	2025-05-04 17:42:15
A113	T15	E22	2025-05-04 18:06:58
A12	T12	E19	2025-04-29 00:42:28
A13	T13	E20	2025-04-29 00:42:28
A14	T14	E21	2025-04-29 00:42:28
A16	T16	E23	2025-04-29 00:42:28

Figure 7.1.1: Task Assignments Table

TaskID	Title	Description	Status	CreatedAt	DueDate	Priority	CreatedBy
T112	Sunday Testing Update	Updated Sunday Testing before Presentation	In Progress	2025-05-04 17:32:32	2025-05-27	Medium	E1
T12	Notification System	Send email notifications for new tasks.	Pending	2025-04-29 00:42:28	2025-06-12	High	E16
T13	Upload Profile Pictures	Allow users to upload and update profile pictures.	Pending	2025-04-29 00:42:28	2025-06-13	Low	E17
T14	Server Deployment	Deploy backend server to AWS EC2 instance.	Pending	2025-04-29 00:42:28	2025-06-14	High	E18
T15	Create Testing Scripts	Automate API tests using Postman or Jest.	Pending	2025-04-29 00:42:28	2025-06-15	Medium	E19
T16	Frontend React Setup	Initialize React app with basic routing.	Pending	2025-04-29 00:42:28	2025-06-16	Medium	E20
T17	Build Redux Store	Setup Redux for state management.	Pending	2025-04-29 00:42:28	2025-06-17	Medium	E21
T18	Handle API Errors	Global error handling for failed API calls.	Pending	2025-04-29 00:42:28	2025-06-18	Medium	E22
T19	Optimize Database Indexes	Add indexes to improve query performance.	Pending	2025-04-29 00:42:28	2025-06-19	High	E23

Figure 7.1.2: Tasks Table

EmployeeID	EmployeeName	Email	Role	SignupDate	Password
E1	John Smith	E1@atm.com	Upper Senior Manager	2025-04-29	$2b$10$gR.h8YnonlrIIdiEyyzSheyA6aD0JEYI.qVSGvTmUOsWQuqMdYnhO
E10	Mia Gonzalez	E10@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E100	Sadie Crawford	E100@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E11	Henry Wilson	E11@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E12	Amelia Anderson	E12@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E13	Alexander Thomas	E13@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E14	Isabella Taylor	E14@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E15	Michael Moore	E15@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW
E16	Charlotte Jackson	E16@atm.com	Employee	2025-04-29	$2b$10$59jr4tozUKiab7iBOV2k..3WJJvtw3.o1Sg.Oqp/zONi8pwuzL5GW

Figure 7.1.3: Employee Table

UpdateID	AssignmentID	UpdateText	UpdateTimestamp
U1	A1	Started working on database schema.	2025-04-29 00:42:28
U10	A10	Employee task view connected with backend.	2025-04-29 00:42:28
U100	A100	Project closure documentation finalized.	2025-04-29 00:42:28
U101	A100	Started working on frontend form.	2025-04-30 09:22:51
U11	A11	Password recovery flow tested.	2025-04-29 00:42:28
U12	A12	Notification system configured.	2025-04-29 00:42:28
U13	A13	Profile picture upload tested.	2025-04-29 00:42:28
U14	A14	Backend server deployed on AWS.	2025-04-29 00:42:28
U16	A16	React app base setup done.	2025-04-29 00:42:28

Figure 7.1.4: Task Status Updates Table

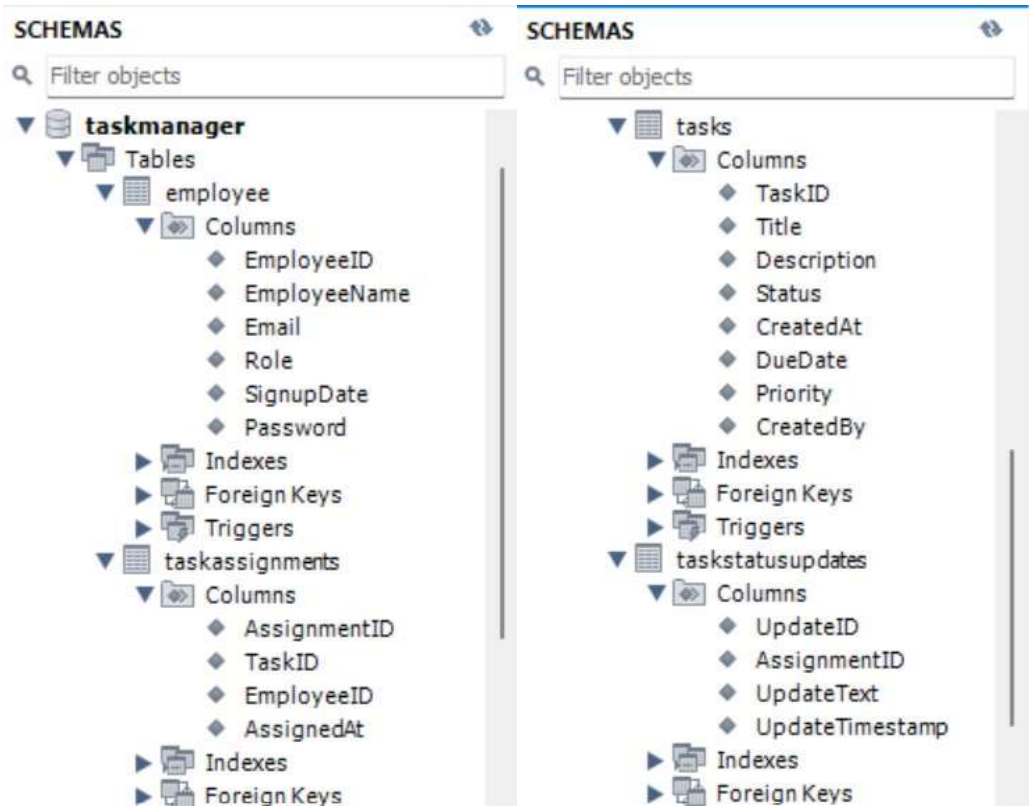


Figure 7.1.5: Schema Design

Figures 7.1.1 through 7.1.4 show all the tables in the database where the actual values are stored. Figure 7.1.5 shows the entire schema for our system within the MYSQL software.

2. User Interface

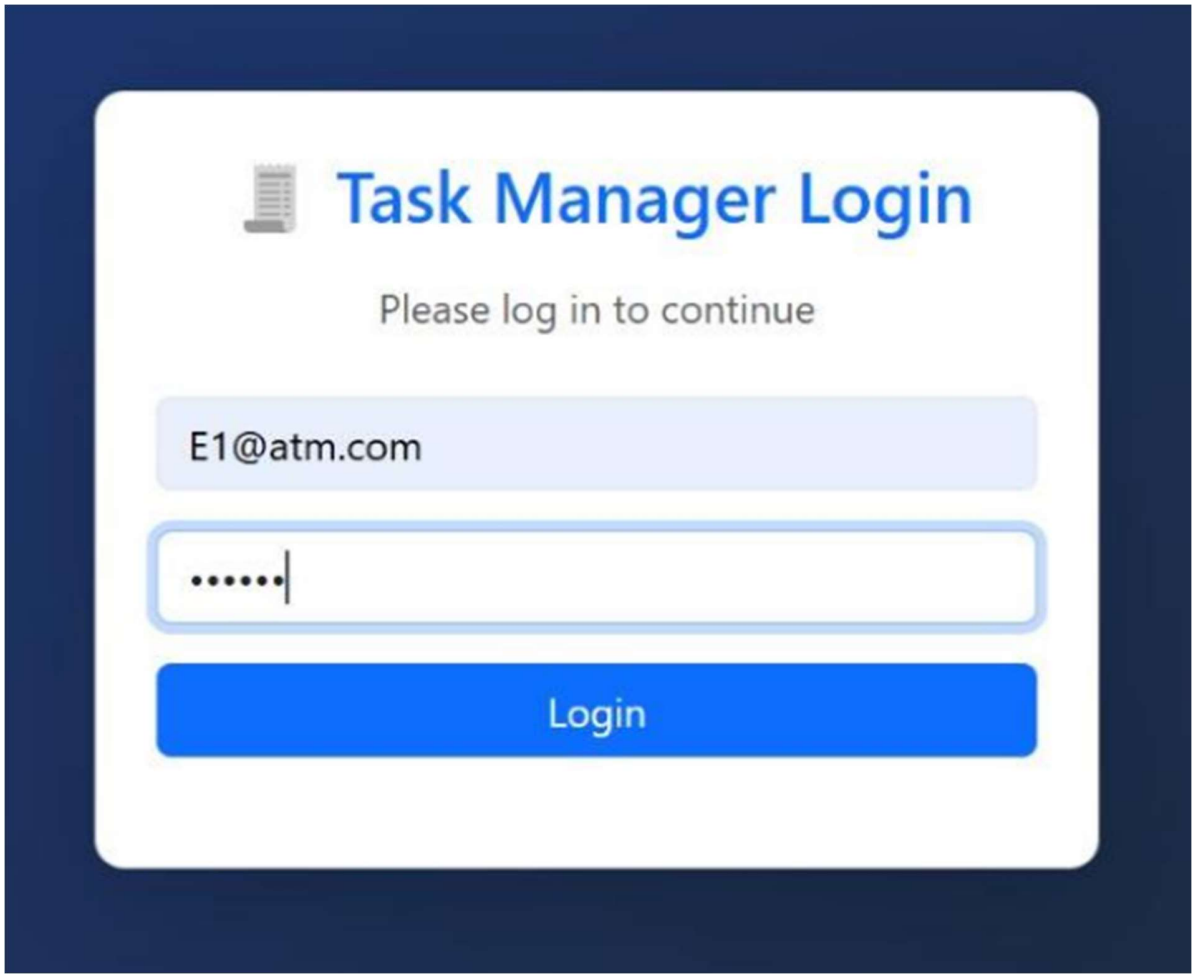
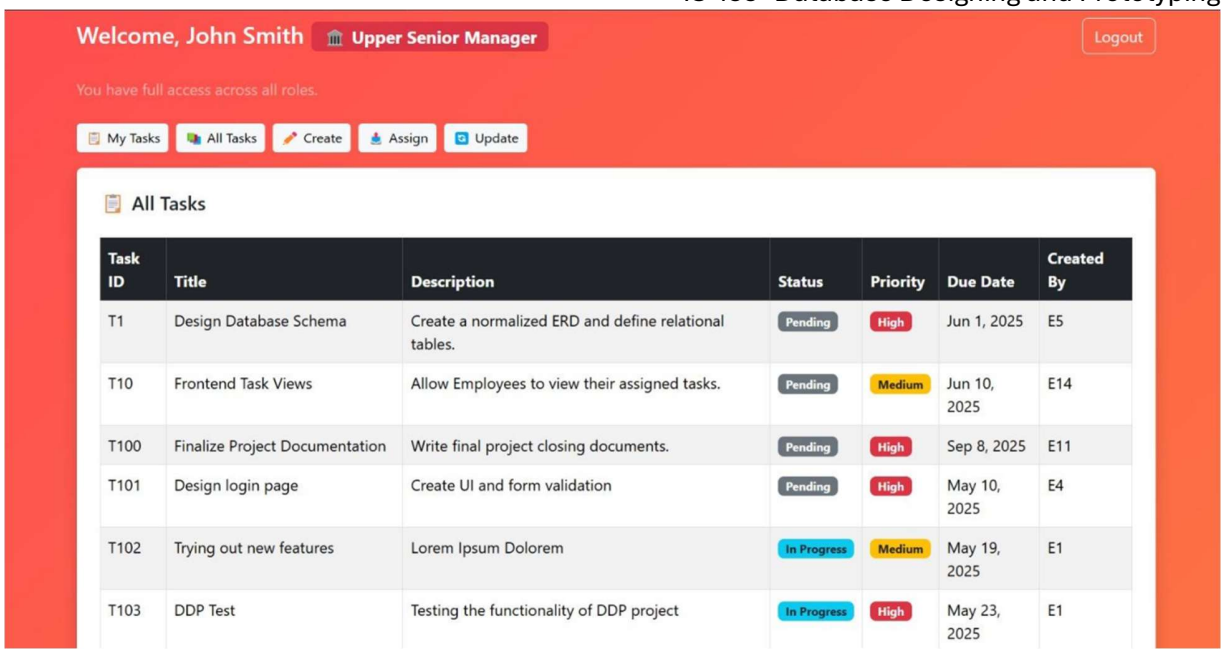


Figure 7.2.1: Employee Login



Task ID	Title	Description	Status	Priority	Due Date	Created By
T1	Design Database Schema	Create a normalized ERD and define relational tables.	Pending	High	Jun 1, 2025	E5
T10	Frontend Task Views	Allow Employees to view their assigned tasks.	Pending	Medium	Jun 10, 2025	E14
T100	Finalize Project Documentation	Write final project closing documents.	Pending	High	Sep 8, 2025	E11
T101	Design login page	Create UI and form validation	Pending	High	May 10, 2025	E4
T102	Trying out new features	Lorem Ipsum Dolorem	In Progress	Medium	May 19, 2025	E1
T103	DDP Test	Testing the functionality of DDP project	In Progress	High	May 23, 2025	E1

Figure 7.2.2: Senior Manager Dashboard

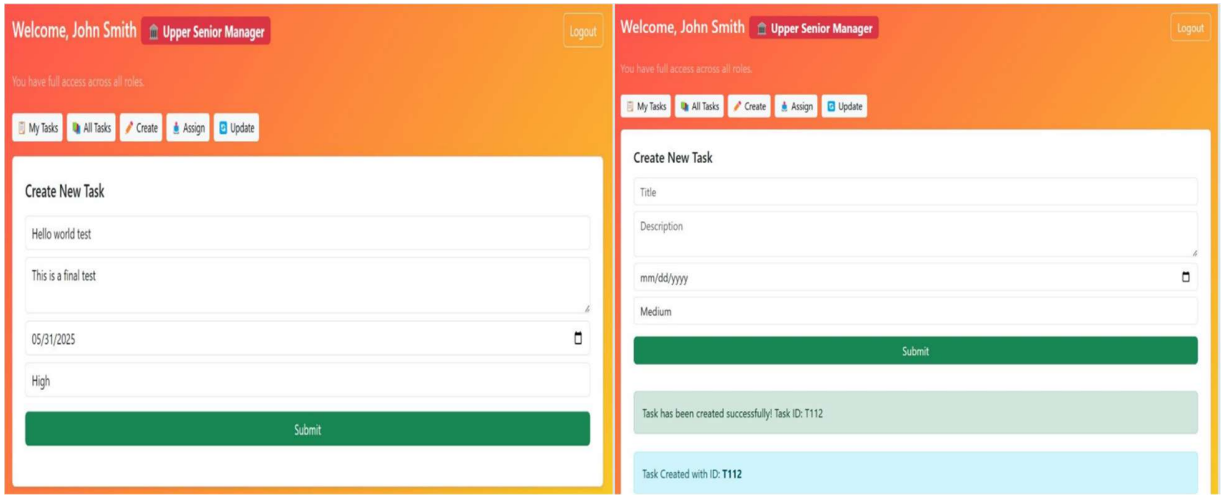


Figure 7.2.3: Task Creation Flow

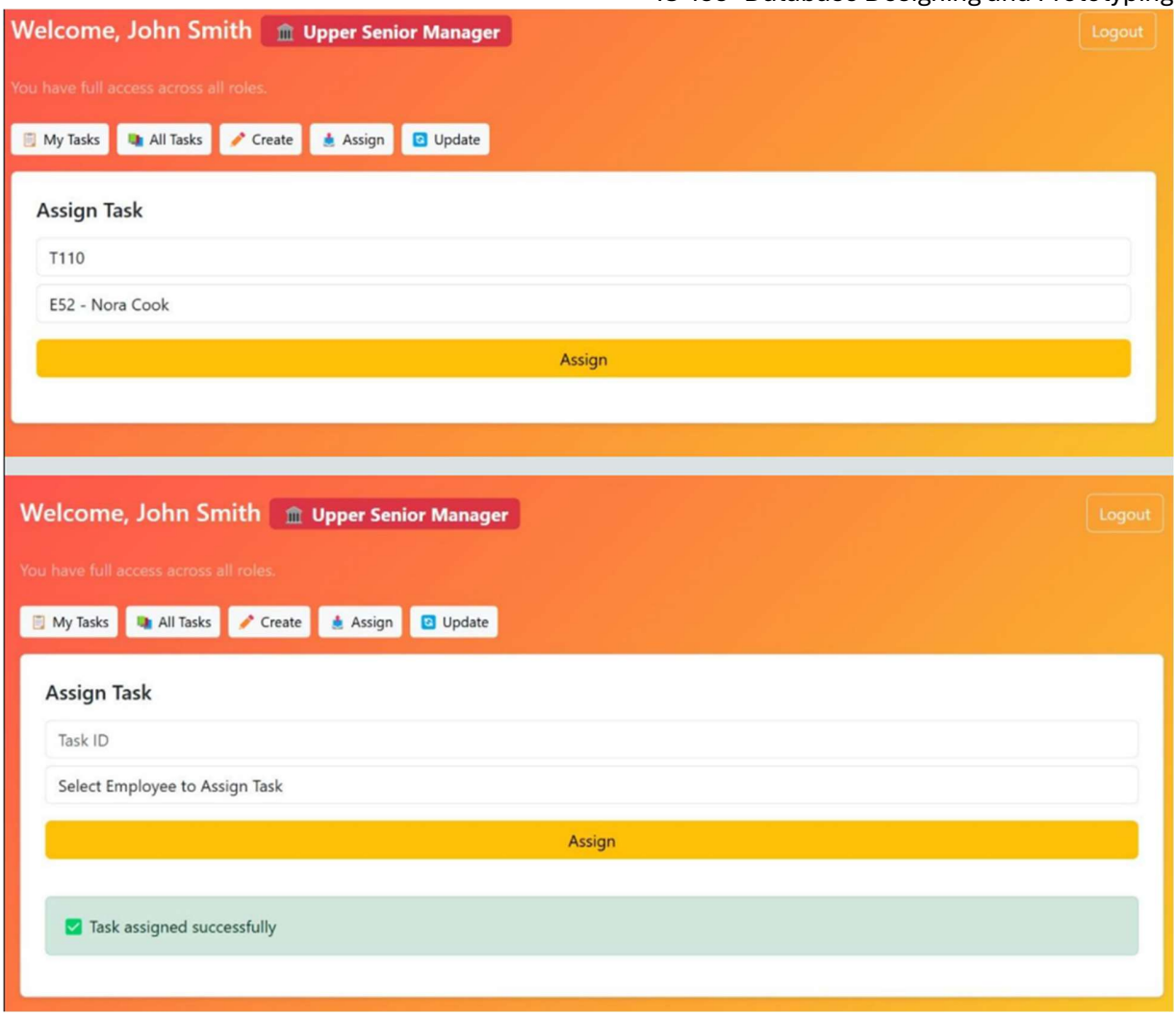


Figure 7.2.4: Task Assignment Flow

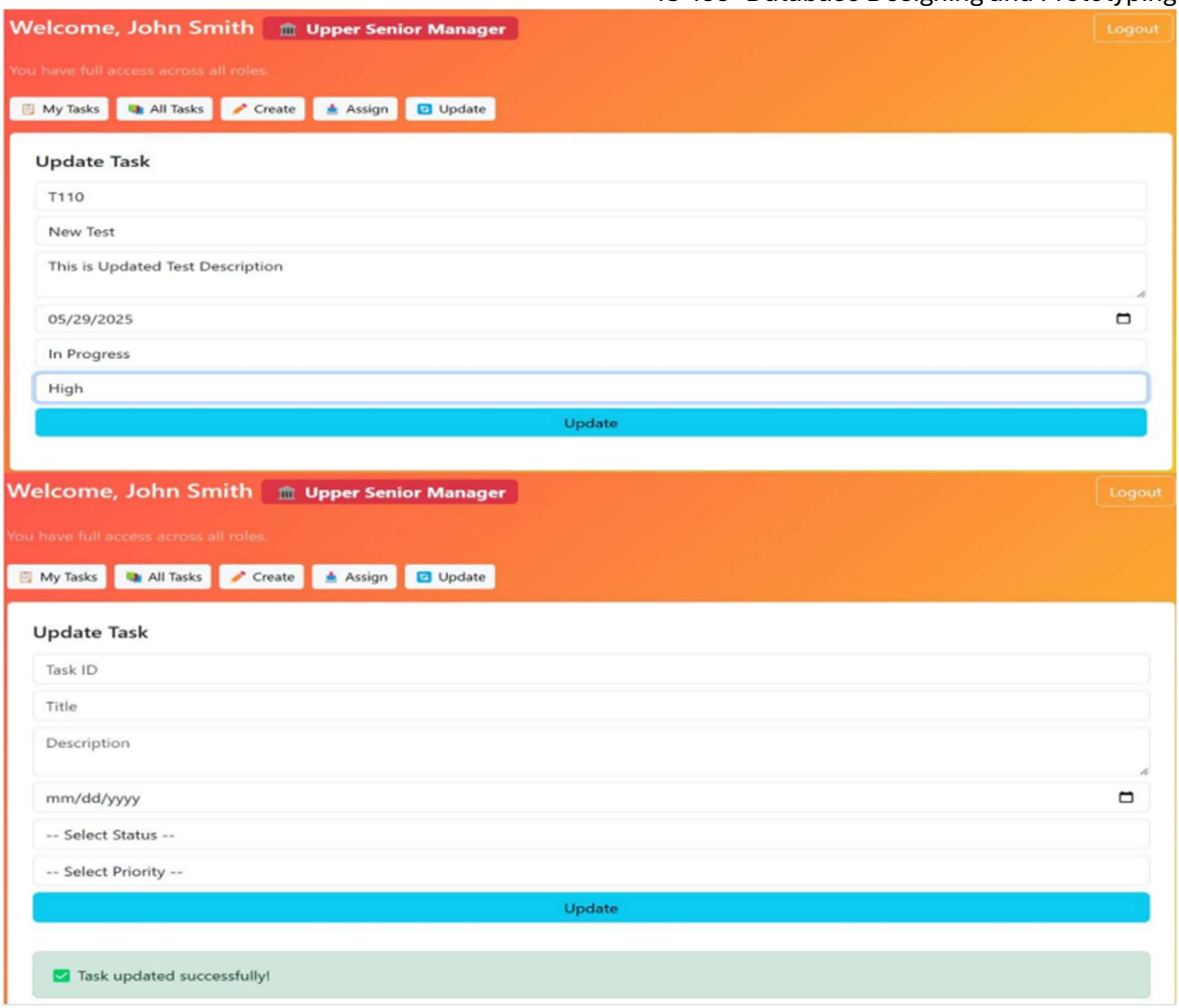


Figure 7.2.5: Task Updating flow

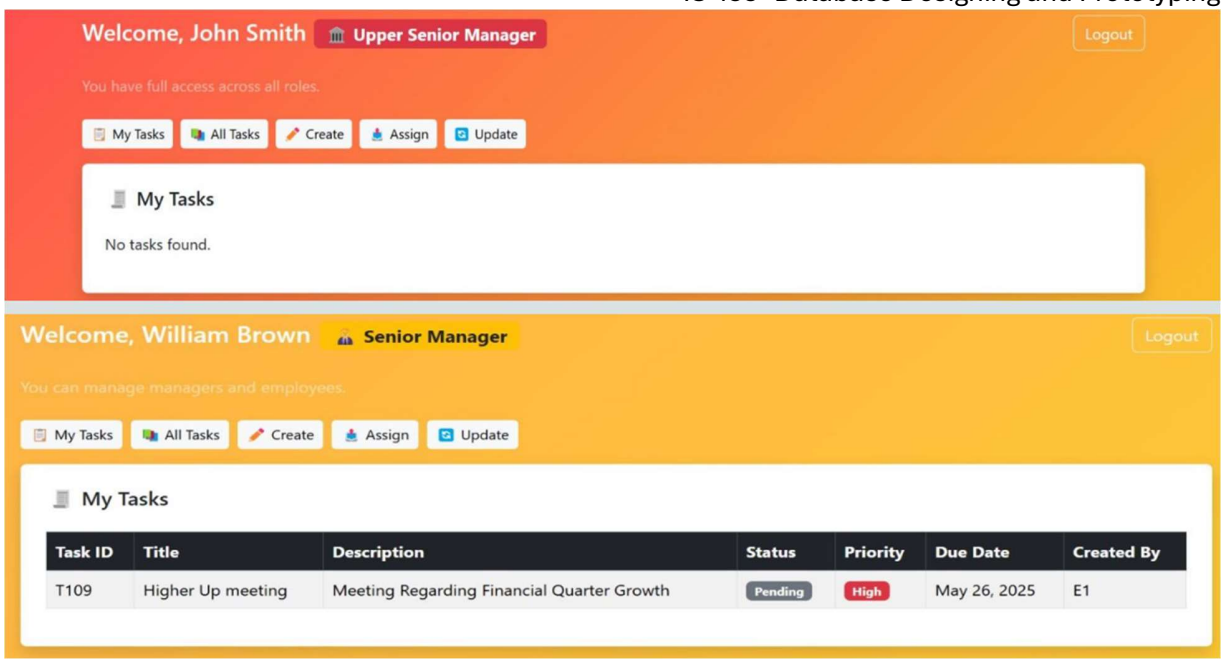


Figure 7.2.6: My Task View for Senior Management

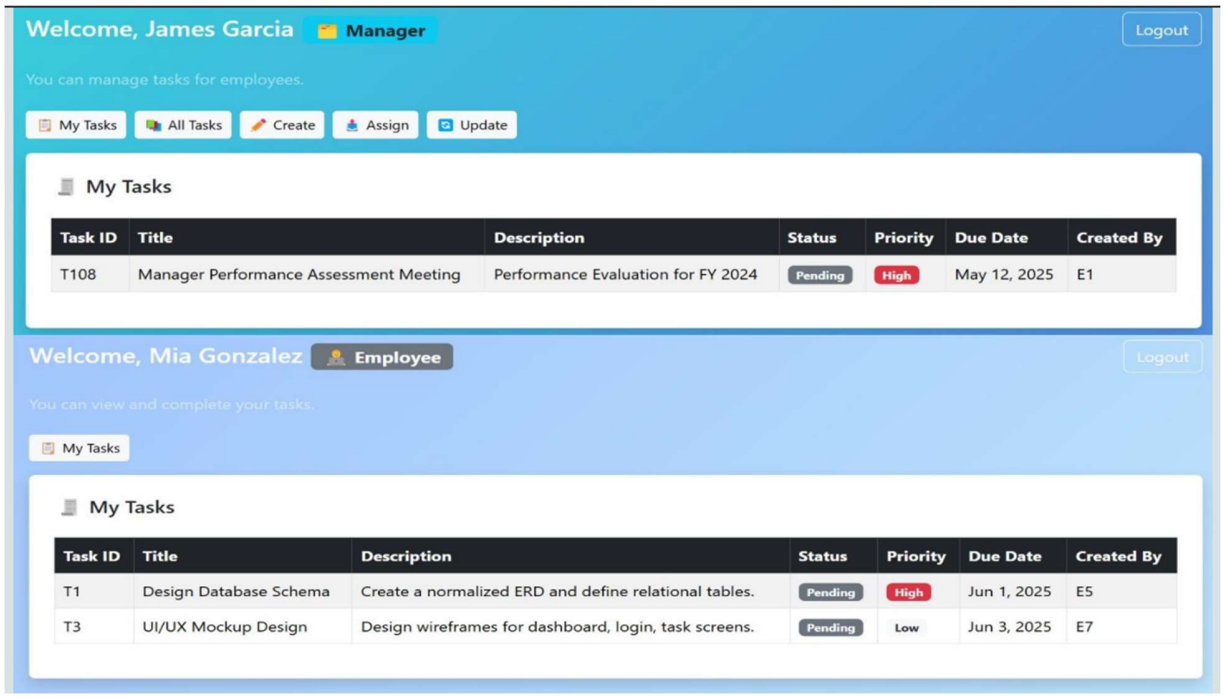


Figure 7.2.7: My Task View for Manager and Employee

Figure 7.2.1 shows the login page, employees put in their login credentials (company mail id and password) to login to their dashboard.

Figure 7.2.2 shows the upper senior manager dashboard that contains all the tasks that have been created in the system.

Figure 7.2.3 shows the task creation flow, the management employee first fills out the details and when they hit submit the green popup gives confirmation that the task has been created.

Similarly, Figure 7.2.4 and Figure 7.2.5 show the task assignment and task updating flow respectively.

Figures 7.2.6 and 7.2.7 show the different views of 'My Tasks' for each role, notice how Upper senior manager has no tasks because no one can assign tasks to upper senior manager, while for other roles they have their respective assigned tasks by their superiors.

3. **Important Code Snippets**

   1. **Task Creation**

```
const createTask = async (req, res) => {
  try {
    const TaskID = await generateId('Tasks', 'TaskID', 'T');
    const { Title, Description, Status, DueDate, Priority, CreatedBy } = req.body;

    await db.query(
      'INSERT INTO Tasks (TaskID, Title, Description, Status, DueDate, Priority,
CreatedBy) VALUES (?, ?, ?, ?, ?, ?, ?)',
      [TaskID, Title, Description, Status, DueDate, Priority, CreatedBy]
    );

    res.status(201).json({ message: 'Task created successfully', taskId: TaskID });
  } catch (error) {
    console.error('Error creating task:', error);
    res.status(500).json({ message: 'Server Error' });
  }
};
```

   2. **Showing tasks to user based on the role the employee is logged in with**

```
const getTasksAssignedTo = async (req, res) => {
```

```
  const { employeeId } = req.params;
  try {
    const [tasks] = await db.query(
      `SELECT t.*
       FROM Tasks t
       JOIN TaskAssignments ta ON t.TaskID = ta.TaskID
       WHERE ta.EmployeeID = ?` ,
      [employeeId]
    );
    res.json(tasks);
  } catch (error) {
    console.error('Error fetching assigned tasks:', error);
    res.status(500).json({ message: 'Server Error' });
  }
};
```

3. **Task Update**

```
const updateTask = async (req, res) => {
  try {
    const { id } = req.params;
    const { Title, Description, Status, DueDate, Priority } = req.body;

    await db.query(
      'UPDATE Tasks SET Title = ?, Description = ?, Status = ?, DueDate = ?, Priority
= ? WHERE TaskID = ?',
      [Title, Description, Status, DueDate, Priority, id]
    );

    res.json({ message: 'Task updated successfully' });
  } catch (error) {
    console.error('Error updating task:', error);
    res.status(500).json({ message: 'Server Error' });
  }
};
```

The above code shows the queries we ran in JavaScript. Whenever the button in the front end was clicked it would hit an API which would run the query to the database and return

the results in the https request. We used try-catch blocks to ensure the system doesn't crash, and that we can pinpoint the point of error and do effective troubleshooting.

8. Conclusion

To conclude, our asynchronous task manager helps solve common problems that large and decentralized organizations face when managing tasks. It makes task assignment more organized by following a clear role hierarchy, where managers can assign tasks only to their team members. The system includes useful features like secure login, task creation, updates, and a role-based user interface that changes based on who is logged in. We also added optional analytics to give users a better view of task progress. Overall, this project improves teamwork, reduces confusion, and helps everyone stay on track with their work in a simple and effective way.